Advanced Threat Detection & Web Security Enhancements

Security Assignment Report

Week 4 — Cybersecurity Lab

Submitted By: Muhammad Shahzaib

DHC-3811

Date: April 2026

1. Introduction

This report documents the implementation of advanced security measures on a Python Flask-based vulnerable web application. The goal of this project was to identify security weaknesses and apply real-world security techniques to harden the application against common attack vectors.

The following three major security enhancements were implemented:

- Intrusion Detection and Real-Time Monitoring
- API Security Hardening (Rate Limiting, CORS, API Key Authentication)
- Security Headers and Content Security Policy (CSP) Implementation

2. Tools & Technologies Used

Tool / Library	Purpose	Version
Python Flask	Web application framework	3.x
Flask-Talisman	Security headers & CSP	1.x
Flask-Limiter	Rate limiting	4.1.1
Flask-CORS	CORS configuration	4.x
monitor.py (custom)	Intrusion detection	Custom
SQLite	Database	Built-in
VS Code	Development environment	Latest

3. Task 1 — Intrusion Detection & Monitoring

3.1 Overview

Since the application runs on Windows 10, traditional Linux-based tools like Fail2Ban and OSSEC are not natively compatible. A custom Python-based intrusion detection monitor (monitor.py) was developed as a Windows-compatible alternative that provides equivalent real-time threat detection functionality.

3.2 How It Works

- The main application (app.py) writes all security events to a log file called security.log using Python's built-in logging module.
- monitor.py runs in a separate terminal and watches security.log in real-time using a tail-like mechanism.
- Each log entry is parsed to extract the event type, IP address, and timestamp.
- If an IP address fails login 5 or more times within 60 seconds, an alert is triggered.
- The monitor also detects rate limit abuse and unauthorized API key attempts.

3.3 Events Logged

Event Type	Trigger	Action
LOGIN_FAILED	Wrong username/password	Count per IP, alert at 5
LOGIN_SUCCESS	Correct credentials	Clear fail count for IP
RATE_LIMIT_EXCEEDED	Too many requests	Print DoS warning
UNAUTHORIZED	Missing/invalid API key	Print API abuse warning
NEW_USER_REGISTERED	New account created	Log registration event

3.4 Test Results

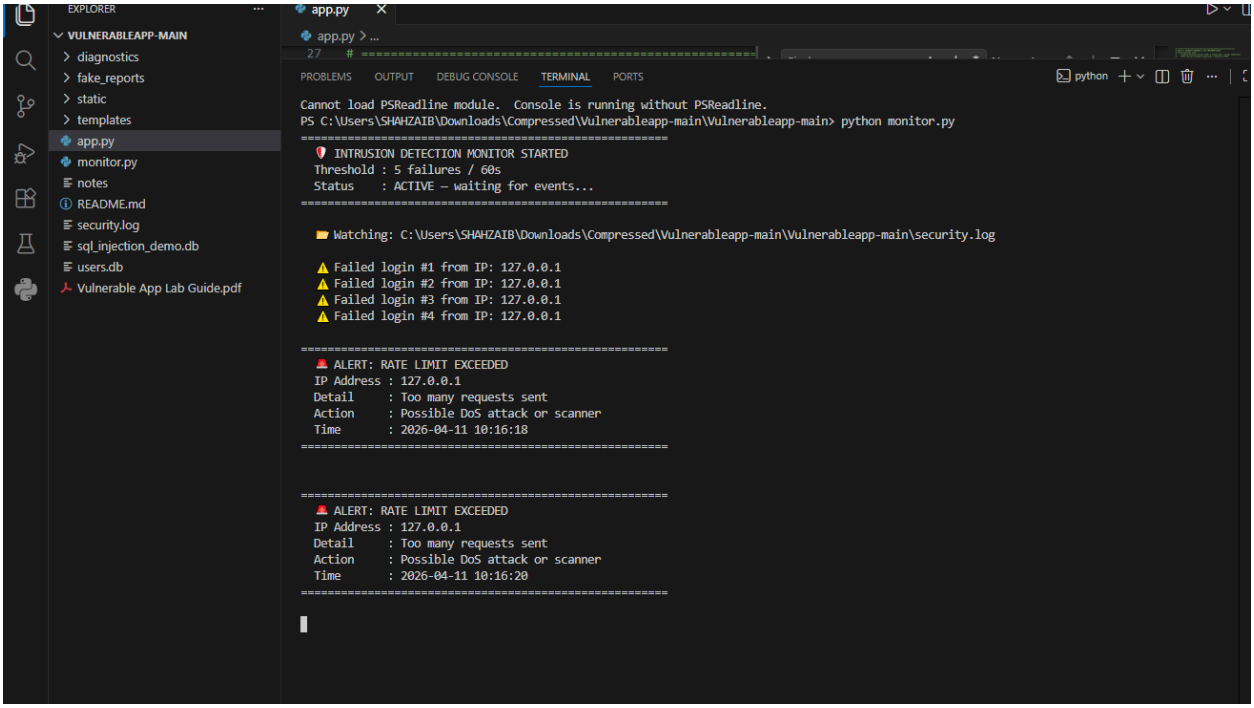
Test Performed: 5 consecutive failed login attempts from the same IP address (127.0.0.1).

Expected Result: Brute force alert printed in monitor terminal after the 5th failed attempt.

Actual Result: Alert triggered successfully as shown below:

```
ALERT: BRUTE FORCE DETECTED | IP: 127.0.0.1 | 5 failed logins in 60 seconds | Action: BLOCK THIS IP
```

Status: PASSED


EXPLORER
VULNERABLEAPP-MAIN
diagnostics
fake_reports
static
templates
app.py
monitor.py
notes
README.md
security.log
sql_injection_demo.db
users.db
Vulnerable App Lab Guide.pdf
app.py
app.py > ...
27 # ==========
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
python
Cannot load PSReadline module. Console is running without PSReadline.
PS C:\Users\SHAHZAIB\Downloads\Compressed\Vulnerableapp-main\Vulnerableapp-main> python monitor.py
==========
INTRUSION DETECTION MONITOR STARTED
Threshold : 5 failures / 60s
Status : ACTIVE — waiting for events...
==========
Watching: C:\Users\SHAHZAIB\Downloads\Compressed\Vulnerableapp-main\Vulnerableapp-main\security.log
Failed login #1 from IP: 127.0.0.1
Failed login #2 from IP: 127.0.0.1
Failed login #3 from IP: 127.0.0.1
Failed login #4 from IP: 127.0.0.1
==========
ALERT: RATE LIMIT EXCEEDED
IP Address : 127.0.0.1
Detail : Too many requests sent
Action : Possible DoS attack or scanner
Time : 2026-04-11 10:16:18
==========
==========
ALERT: RATE LIMIT EXCEEDED
IP Address : 127.0.0.1
Detail : Too many requests sent
Action : Possible DoS attack or scanner
Time : 2026-04-11 10:16:20
==========

4. Task 2 — API Security Hardening

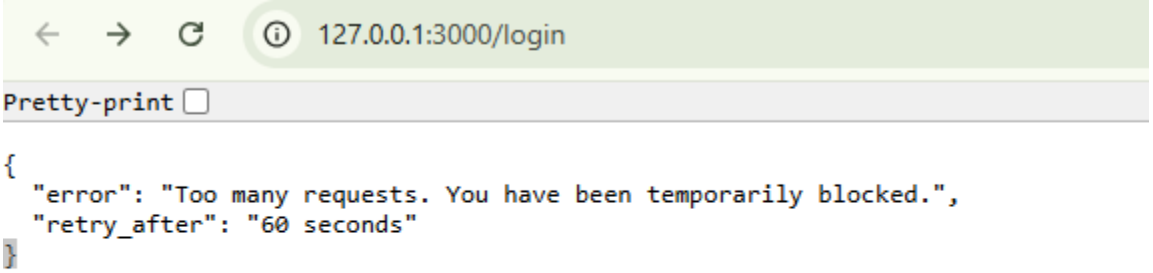
4.1 Rate Limiting

Flask-Limiter was used to restrict the number of requests per IP address within a given time window. This prevents brute-force and denial-of-service attacks.

Endpoint	Limit Applied	Purpose
/login	5 per minute	Prevent brute force
/register	5 per hour	Prevent spam accounts
/search	20 per minute	Prevent enumeration
/system_info	10 per minute	Protect sensitive endpoint
Global	200/day, 50/hour	Overall protection

Test Result: After 5 rapid login attempts, the server returned HTTP 429 Too Many Requests with the message: 'Too many requests. You have been temporarily blocked.'

Status: PASSED



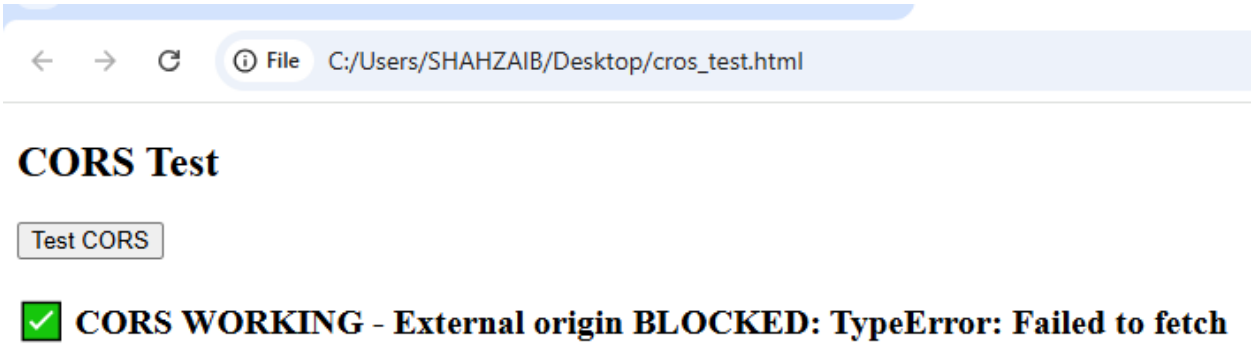
4.2 CORS Configuration

Flask-CORS was configured to restrict API access to authorized origins only. Requests coming from unknown origins (such as malicious external websites) are automatically blocked by the browser.

- Allowed Origin: http://localhost only
- Allowed Methods: GET, POST
- Allowed Headers: Content-Type, X-API-Key

Test Result: A test HTML file opened from the Desktop (different origin) attempted to call the API. The browser blocked the request and returned: 'CORS WORKING - External origin BLOCKED: TypeError: Failed to fetch'

Status: PASSED



4.3 API Key Authentication

Sensitive endpoints (/system_info and /os_info) were protected with API key authentication. A custom decorator require_api_key was implemented that checks the X-API-Key request header before allowing access.

Without API Key:

```
GET /system_info  →  401 Unauthorized: {"error": "Unauthorized. Valid API key required."}
```

With Valid API Key:

```
GET /system_info (X-API-Key: mysecretapikey123)  →  200 OK: {"output": "pc-name"}
```

Status: PASSED

With Out API Key:

```
PS C:\Users\SHAHZAIB\Downloads\Compressed\Vulnerableapp-main\Vulnerableapp-main> curl http://localhost:3000/system_info?cmd=whoami


StatusCode        : 200
StatusDescription : OK
Content           : <!DOCTYPE html>
                    <html>
                    <head>
                        <meta charset="UTF-8">
                        <meta name="viewport" content="width=device-width, initial-scale=1.0">
                        <title>Login</title>
                        <style>
                            body {
                                font...
RawContent        : HTTP/1.1 200 OK
                    Access-Control-Allow-Origin: http://localhost
                    Vary: Origin
                    Permissions-Policy: browsing-topics=()
                    X-Frame-Options: SAMEORIGIN
                    X-Content-Type-Options: nosniff
                    Content-Security-Pol...
Forms             : {}
Headers           : {[Access-Control-Allow-Origin, http://localhost], [Vary, Origin], [Permissions-Policy, browsing-topics=()],
                    [X-Frame-Options, SAMEORIGIN]...}
Images            : {@{innerHTML=; innerText=; outerHTML=<IMG alt=Logo src="/static/NGS-logo.png">; outerText=; tagName=IMG; alt=Logo;
                    src=/static/NGS-logo.png}}
InputFields       : {@{innerHTML=; innerText=; outerHTML=<INPUT name=username required>; outerText=; tagName=INPUT; name=username;
                    required=}, @{innerHTML=; innerText=; outerHTML=<INPUT type=password value="" name=password required>; outerText=;
                    tagName=INPUT; type=password; value=; name=password; required=}}
Links             : {@{innerHTML=Register; innerText=Register; outerHTML=<A href="/register">Register</A>; outerText=Register; tagName=A;
                    href=/register}}
ParsedHtml        : mshtml.HTMLDocumentClass
RawContentLength  : 3737
```

With API Key:

```
PS C:\Users\SHAHZAIB\Downloads\Compressed\Vulnerableapp-main\Vulnerableapp-main> Invoke-WebRequest -Uri "http://localhost:3000/system_i
md=whoami" -Headers @{"X-API-Key"="mysecretapikey123"}


StatusCode        : 200
StatusDescription : OK
Content           : {
                      "output": "lenovo\\shahzaib\n"
                    }

RawContent        : HTTP/1.1 200 OK
                    Access-Control-Allow-Origin: http://localhost
                    Vary: Origin
                    Permissions-Policy: browsing-topics=()
                    X-Frame-Options: SAMEORIGIN
                    X-Content-Type-Options: nosniff
                    Content-Security-Pol...
Forms             : {}
Headers           : {[Access-Control-Allow-Origin, http://localhost], [Vary, Origin], [Permissions-Policy, browsing-topics=()],
                    [X-Frame-Options, SAMEORIGIN]...}
Images            : {}
InputFields       : {}
Links             : {}
ParsedHtml        : mshtml.HTMLDocumentClass
RawContentLength  : 37



PS C:\Users\SHAHZAIB\Downloads\Compressed\Vulnerableapp-main\Vulnerableapp-main>
```

5. Task 3 — Security Headers & CSP Implementation

5.1 Overview

Flask-Talisman was used to automatically add security headers to every HTTP response. These headers instruct the browser to enforce additional security policies that protect users from common web attacks.

5.2 Headers Implemented

Header	Value	Protection Against
Content-Security-Policy	default-src 'self'	XSS / Script Injection
Strict-Transport-Security	max-age=31536000	Man-in-Middle Attack
X-Content-Type-Options	nosniff	MIME Sniffing Attack
X-Frame-Options	SAMEORIGIN	Clickjacking Attack
Referrer-Policy	strict-origin-when-cross-origin	Information Leakage
Access-Control-Allow-Origin	http://localhost	Unauthorized CORS

5.3 Content Security Policy (CSP) Details

The CSP was configured to only allow resources from the same origin ('self'), effectively blocking all external scripts, stylesheets, and media that could be injected by an attacker.

CSP Directive	Value	Meaning
default-src	'self'	All resources from same origin only
script-src	'self'	No inline or external scripts
style-src	'self' 'unsafe-inline'	Styles from same origin
img-src	'self' data:	Images from same origin or data URIs
object-src	'none'	Block all plugins and embeds
frame-src	'none'	Block all iframes

Test Result: Headers were verified using Chrome Developer Tools (F12 > Network > Response Headers). All headers were present and correctly configured.

Status: PASSED

Elements Console Sources Network 3

Preserve log Disable cache No throttling

Filter Invert More filters

All Fetch/XHR Doc CSS JS Font Img Media Manifest Socket Wasm Other

1,000 ms 2,000 ms 3,000 ms 4,000 ms 5,000 ms 6,000 ms 7,000 ms

Name
login
NGS-logo.png
local-storage.js
util.js
fte-utils.js
express-fte.js
express-utils.js
main.tsx-CtzEbR1O.js
main.tsx-BKVWIQVK.js
client-BGioy6-K.js
index-DA4uJ5Kf.js
webcomponents-bundle-Cazg...
index-Cmeg3mF-.js
index-HIOnbfQe.js
index-DdubJ4IQ.js
index-DefbWwKa.js
sidePanelUtil.js
readability.js
favicon.ico
login
FloatingActionButton.js
vendor.js
otherVendors.js

Headers Preview Response Initiator

▼ General

Request URL	http://127.0.0.1:3000/login
Request Method	GET
Status Code	200 OK
Remote Address	127.0.0.1:3000
Referrer Policy	strict-origin-when-cross-origin

▼ Response headers Raw

Access-Control-Allow-Origin	http://localhost
Connection	close
Content-Length	3737
Content-Security-Policy	default-src 'self'; script-src 'self'; style-src 'self' 'unsafe-inline'; img-src 'self' data:; font-src 'self'; object-src 'none'; frame-src 'none'
Content-Type	text/html; charset=utf-8
Date	Sat, 11 Apr 2026 07:51:23 GMT
Permissions-Policy	browsing-topics=()
Referrer-Policy	strict-origin-when-cross-origin
Server	Werkzeug/3.0.1 Python/3.14.2
Vary	Origin
X-Content-Type-Options	nosniff

6. Final Summary — All Tests

#	Feature	Fix Applied	Test Method	Result
1	Brute Force Detection	monitor.py real-time	5 failed logins	PASSED
2	Rate Limiting	Flask-Limiter 5/min	Rapid login clicks	PASSED
3	API Key Auth	X-API-Key header	curl with/without key	PASSED
4	CORS Restriction	Flask-CORS localhost	External HTML file	PASSED
5	Security Headers	Flask-Talisman	F12 Network tab	PASSED
6	CSP	Talisman CSP config	F12 Response Headers	PASSED

7. Conclusion

This project successfully demonstrated the implementation of multiple layers of web application security on a Python Flask application. Each security measure was tested and verified to be functioning correctly.

The key outcomes of this project were:

- A custom intrusion detection system was built that works on Windows and detects brute force attacks in real-time.
- API endpoints were hardened with rate limiting, CORS policies, and API key authentication to prevent unauthorized access.
- Security headers including CSP and HSTS were implemented to protect users from XSS, clickjacking, and man-in-the-middle attacks.
- All implemented security measures were tested and produced the expected results.

This project provided hands-on experience with real-world web security techniques commonly used in professional software development environments.